



CC91 校园论坛系统 — 缺陷反思与课程建议

项目： CC91 校园论坛系统 (Spring Boot 微服务 + React 19)

周期： 2026 年 4 月 – 2026 年 6 月

团队： 11 人，采用 Lead / Developer / QA 三智能体协作模式

第一部分：缺陷与反思

1.1 正向经验

本项目采用 Claude Code 作为 Lead，下设 Developer 和 QA 两个 Subagent，形成五阶段 workflow。经过三个月实践，以下方面表现突出：

- **任务结构化管理降低遗漏风险。** 每个任务包含唯一 ID、背景目标、实现范围、验收标准和依赖关系，11 人团队在同一框架下协作，任务边界清晰、交接成本低。
- **Skills 体系让 AI 输出质量可控。** 通过为 Developer 绑定 5 个技能模块，AI 生成的代码在安全性、分层架构、错误处理等方面保持一致水准，减少了人工审查的修正成本。
- **QA 智能体的“强制运行测试”规则有效。** [qa.md](#) 明确禁止 QA 仅做静态分析，要求必须实际执行测试并输出报告，累计补齐了 7 个服务共 685 个测试用例。
- **CodeGraph MCP 显著提升代码导航效率。** 索引 308 个文件、4284 个符号、6998 条边，替代了传统 grep/read 循环，在微服务拆分阶段发挥了关键作用。

1.2 暴露的问题与反思

问题一：AI 对“隐性上下文”的认知盲区

压力测试第四轮 scenario-5 出现 93.5% 的失败率，根因是 PostService 写入路径通过 Feign 同步调用 UserService 获取 userId——AI 正确实现了 Feign 调用，但未意识到“在写入热路径上同步调用远程服务获取本可从 JWT 中解析的数据”是架构层面的性能反模式。AI 在局部代码正确性上表现良好，但在系统级横切关注点（调用链延迟、资源竞争、故障传播）上缺乏全局视角。

反思： 审查清单需要增加"关键路径调用链审计"维度，要求对每个跨服务同步调用标注延迟预算和降级策略。

问题二：多智能体间的信息衰减

Lead → Developer → QA 的消息传递链中，每次 SendMessage 都产生信息损失。例如 Lead 在任务描述中写明了文件类型白名单（JPEG/PNG/WebP），Developer 漏掉了 WebP 校验，QA 仅按"常见场景"测试了 JPEG/PNG。边界条件在传递中逐级丢失。

反思： 验收标准中的"边界验收"应从自由文本改为结构化 checklist（如"逐一验证每个允许类型 + 至少 3 个禁止类型"），减少解释空间，让 Developer 和 QA 无法选择性忽略。

问题三：AI 生成代码的"过度自信"

UserServiceClientFallback 中 `getUserByUsername` 的 fallback 最初返回了 `id=null` 的占位对象而非 `null`，导致调用方 NPE。该问题在单体架构下不会暴露，迁移到微服务后压测中才暴露（313 次 `author_id null` 错误）。AI 倾向于构造"看起来合理"的 fallback 行为，而非从契约角度思考"调用失败时调用方期望什么"。

反思： Developer Agent 配置中需增加"Feign Fallback 契约原则"：列表返回空列表，单对象返回 `null` 并让调用方显式处理，避免在 fallback 中"制造数据"。

问题四：人机协作的权责错位

项目中"人的角色"（11 位成员按软件工程角色分工）和"智能体的角色"（Lead / Developer / QA）存在错位。例如 HikariCP 连接池大小由 DevOps 工程师初始设为 50，AI 生成配置时未质疑，直到压测第一轮才发现 50 连接在 500 并发下瞬间打满，经过五轮调参才确定最终值。Lead 智能体的审查清单缺少对"人的决策"的审查入口——当人的配置决策或范围决策出现偏差时，AI 不会主动提出异议。

反思： AI 适合执行"已知规则的检查"，而"未知规则的发现"仍需要人的判断。建议在 [CLAUDE.md](#) 中增加"人工决策清单"——哪些决策必须由人确认后才能进入开发，与 AI 自动审查形成互补。

第二部分：课程建议

建议：增加"AI 辅助软件工程"的独立模块

当前课程内容聚焦于传统软件工程实践（需求分析、架构设计、编码、测试、部署），AI 编程工具（Copilot、Cursor、Claude Code 等）通常作为"辅助手段"分散在各环节中，缺乏系统性的讲解。学生在项目中"边做边学"，大量时间花在试错上——如何编写有效的 Agent 配置、如何设计合理的审查清单、如何选择和使用 Skills，这些方法论如果能前置讲授，开发效率和代码质量都会有显著提升。

建议增设一个 2-4 学时的独立模块，具体内容如下：

1. AI Agent 的工作原理与局限性（1 学时）

多数学生对 AI 编程工具的认知停留在"它能写代码"，但对其工作机理缺乏理解，导致两种极端：要么盲目信任 AI 的输出，要么完全不信任而手工编写所有代码。

应覆盖的核心知识点：

- **Token 窗口与上下文衰减：** AI 不是在"读完整个项目"，而是在有限的 Token 窗口内工作。理解这一点，学生才会明白为什么"把整个需求文档丢给 AI"效果不好，而"拆成小任务、每个任务附带精准的代码上下文"效果好。长对话中上下文逐渐被压缩和截断，早期信息会丢失，因此单次任务不宜过大。
- **幻觉与过度自信：** AI 倾向于构造"看起来合理"的输出，而非承认不确定。例如生成一个"看起来能跑"的异常处理、一个"看起来完整"的 fallback 逻辑，但在特定边界条件下会导致运行时错误。应通过典型案例让学生建立"AI 输出需要验证"的肌肉记忆。
- **工具调用的确定性边界：** AI 通过工具（Bash、Read、Grep、代码索引等）与环境交互，但工具返回的信息是有限的。AI 能正确实现一个函数接口，但它无法自动感知"这个接口在高并发下是否会成为瓶颈"——这超出了工具调用的信息边界。学生需要理解 AI 的"感知范围"受限于工具返回的数据，不能期待 AI 发现它"看不到"的问题。
- **结构化代码索引的价值：** 对比传统文本搜索和符号级代码知识图谱，让学生理解为什么工程化的代码索引（能区分函数定义/调用关系/类型层级）比纯文本搜索更能帮助 AI 准确理解代码结构。

2. Agent 配置的工程化方法（1 学时）

这是当前课程最缺失的部分。学生需要学习如何将团队经验固化为可执行的 Agent 配置，而非每次从零开始口头描述需求。

(1) 项目级配置文件的编写原则：

- 不是"写文档"，而是"写协议"——配置文件定义 AI 的行为边界和 workflows，应该是精确、可验证的约束，而不是模糊的指导思想。
- 核心要素：团队角色定义（谁做什么）、工作流阶段（每个阶段的入口/出口条件）、审查清单（具体的检查项而非泛泛的"注意安全"）、技术栈声明。
- 常见反模式：把配置文件写成 README 式的项目介绍，AI 不需要知道"这是一个使用某某技术栈的项目"，它需要的是"你期望我怎么工作、不能做什么"。
- 工作流分解：将开发过程拆分为有明确完成标准的阶段（如理解规划 → 开发 → 测试 → 审查 → 收尾），每个阶段定义入口条件和交付物，避免 AI 在不同阶段之间跳跃。

(2) Subagent 的角色协议设计：

- Developer Agent：核心职责（实现代码 + 补充测试）、绑定的 Skills 及其适用场景、自查清单（提交前必须验证的事项）。
- QA Agent：核心红线（禁止只做静态分析、必须实际运行测试）、测试优先级（先跑已有测试确认无回归 → 再跑新增测试）、报告格式（通过/失败/复现步骤/环境信息）。
- 关键原则：Agent 协议不是"能力描述"，而是"约束声明"——定义什么不能做比定义什么能做更重要。例如"禁止只做静态分析"比"请多做测试"有效得多。

(3) Skills 的选择与组合：

- Skills 的可组合性：一个 Developer Agent + 多个 Skills 比创建多个 Developer Agent 更经济和可维护。Skills 是知识模块，Agent 是执行者，两者应解耦。
- 版本锁定的权衡：锁定版本保证可回溯性，但也可能导致"技能腐化"——锁定的最佳实践随时间推移可能不再是"最佳"。需要建立版本健康检查的习惯，定期对比锁定版本与上游的变更。
- 选型原则：不是 Skills 越多越好。应按"每个 Skill 解决一类可明确界定的问题"来筛选，避免功能重叠和上下文浪费。

3. MCP 协议的选型与实践（1 学时）

MCP（Model Context Protocol）是 AI Agent 连接外部工具的标准接口，但学生需要学会"什么

时候值得引入一个 MCP Server，什么时候用内置工具就够了”。

- **代码知识图谱类：** 适合需要频繁代码导航的大型项目。收益是替代 grep/read 循环，提供亚毫秒级符号搜索和调用链追踪。成本是需要初始化索引。
- **浏览器自动化类：** 适合需要 E2E 测试和 UI 验证的项目。收益是 QA Agent 可以像真人一样操作页面、根据页面状态做动态决策。成本是需要浏览器环境。
- **选型决策框架：** 如果终端命令已经能完成（如运行测试、git 操作、文件操作），不需要 MCP。如果任务需要“运行时的动态决策”（如“根据页面当前状态决定点击哪个按钮”、“根据代码结构决定修改范围”），才值得引入。核心判断标准：任务是否需要 AI 在工具返回的信息基础上做二次判断。

4. 课堂案例讨论（1 学时）

用真实项目数据组织讨论，让学生在具体场景中理解 AI 协作的决策逻辑：

- **案例方向一：性能调优中的 AI 协作。** 讨论链路——AI 能提供数据分析（“连接池使用率 100%，建议扩容”），但根因判断（是配置不够还是架构有问题）和跨层面问题定位（客户端端口耗尽等超出服务端工具感知范围的问题）需要人的判断介入。让学生讨论：AI 分别在哪些环节做得好、哪些环节需要人介入，画出“人机协作边界”。
- **案例方向二：大规模代码生成中的质量控制。** AI 能在给定架构边界后批量完成代码拆分、接口生成和配置迁移，但也可能系统性引入同类设计缺陷（如 fallback 逻辑在所有服务中采用了相同的不安全模式），这些缺陷在简单场景下不可见，复杂场景下集中暴露。讨论：如何在 AI 大批量生成代码时设置“安全检查点”，哪些检查可以自动化、哪些必须人工抽查。